Playwright Capstone Project

OrangeHRM Automation Testing Report

Field	Details
Author	Shiny Belsiya R
Application Under Test	OrangeHRM Demo HR Management System
Application URL	https://opensource-demo.orangehrmlive.com/
Automation Framework	Playwright with JavaScript and Page Object Model
Planned Coverage	8 service modules, 120 planned scenarios

Project Overview

This capstone report defines a professional automation testing plan for the **OrangeHRM Human Resource Management System** using **Playwright**. The project is planned as a QA automation framework rather than an application development activity. The primary objective is to validate critical business workflows available within the **Authentication, Admin, Leave, My Info, Performance, PIM, Recruitment, and Time** modules through structured automation testing.

The framework follows the **Page Object Model** architecture so that automation scripts remain reusable, maintainable, and easier to update when application changes occur. Since OrangeHRM is a Human Resource Management platform containing employee records, attendance information, recruitment processes, leave management, and performance tracking activities, the project focuses on validating these functionalities through safe and reliable automation practices.

The completed framework is expected to improve regression testing efficiency, reduce repetitive manual testing effort, and provide reusable automation assets for future testing activities. The project also aims to provide structured execution evidence and support consistent validation of critical Human Resource Management workflows.

Website Selection Rationale

OrangeHRM was selected because it is a widely used Human Resource Management application that contains multiple business workflows such as **employee management, leave processing, recruitment, attendance tracking, performance monitoring, and administrative operations.** These functionalities provide suitable automation coverage for demonstrating end-to-end testing activities.

The application contains dynamic user interface elements including **dropdowns, date pickers, search filters, validation messages, and workflow-driven forms**. These features allow the implementation of realistic automation scenarios while validating critical Human Resource processes.

OrangeHRM also provides a structured environment for validating business workflows without requiring backend modifications or database access. Its modular design makes it an ideal application for demonstrating automation framework implementation, regression testing, and workflow validation using **Playwright**.

Project Objectives

- Prepare a structured automation testing framework for **OrangeHRM** using **Playwright** and JavaScript.
- Design module-wise automation coverage across **Authentication, Admin, Leave, My Info, Performance, PIM, Recruitment, and Time** modules.
- Implement **Page Object Model** architecture to improve script maintainability, readability, and reusability.
- Generate execution evidence through **Allure Reports**, **Playwright HTML Reports**, screenshots, traces, and CI/CD workflow logs.
- Perform **Cross-Browser Testing** across supported browsers to validate consistent application behavior.
- Improve regression testing efficiency by reducing repetitive manual validation activities.
- Create reusable automation assets that can support future framework enhancements and additional test coverage.
- Establish a scalable automation framework capable of validating critical Human Resource Management workflows effectively.

Technology Stack

Technology	Purpose	Usage in Project
Playwright	Browser Automation	Automates UI workflows, assertions, cross-browser execution, traces, and screenshots.
JavaScript	Scripting Language	Used for writing test cases, page objects, reusable helper functions, and validations.
Node.js	Runtime Environment	Runs Playwright tests and manages packages through npm.
Page Object Model	Framework Pattern	Separates page locators and actions from test scripts for maintainability.
Git and GitHub	Version Control	Stores project code, documentation, and workflow files in a repository.
GitHub Actions	CI /CD Execution	Runs automated tests and uploads reports after push or pull request events.
Allure + Playwright HTML Report	Execution Evidence	Provides pass/fail status, duration, test details, and failure information.
Trace Viewer	Debugging	Shows step for execution, screenshots, action for failures.

The selected technology stack provides a reliable foundation for implementing a scalable automation framework. **Playwright** supports efficient browser automation and cross-browser execution, while **JavaScript** enables flexible test development. **GitHub Actions** helps automate test execution, and **Allure Reports** provide detailed reporting and execution insights. Together, these technologies support maintainable automation and efficient validation of OrangeHRM workflows.

Scope of Testing

Category	Description
In Scope	Authentication, Admin, Leave, My Info, Performance, PIM, Recruitment, and Time modules.
Out of Scope	Database validation, security testing, load testing, backend code modification, and infrastructure validation.
Functional Testing	Validation of business workflows and module functionalities.
Regression Testing	Re-execution of critical workflows after framework updates.
Cross-Browser Testing	Validation across Chromium, Firefox, and WebKit browsers.
UI Validation	Forms, search filters, dropdowns, validations, and navigation workflows.
CI/CD Validation	Verification of GitHub Actions workflow execution and report generation.

The scope of this project focuses on automating major business workflows available within the OrangeHRM application. Testing activities include employee management, leave processing, recruitment operations, attendance tracking, performance monitoring, and user administration workflows. The framework validates application behavior from an end-user perspective through structured automation scenarios.

The project does not include backend database validation, security assessments, performance testing, or application source code modifications. The primary objective is to ensure that critical Human Resource Management functionalities operate correctly and consistently across supported browser environments.

The planned automation coverage provides comprehensive validation of selected modules while maintaining safe testing boundaries. This approach helps improve regression testing efficiency and ensures reliable verification of business-critical processes.

Services / Modules Planned

Service / Module	Coverage Focus	Cases	Severity
M1 - Authentication	Login, Logout, Session Validation, Access Control.	15	Critical

M2 - Admin	User Management, Search Operations, Filters, Administrative Validation.	15	Critical
M3 - Leave	Leave Requests, Leave Lists, Leave Assignment, Status Validation.	15	High
M4 - My Info	Personal Information, Contact Details, Qualifications, Memberships.	15	High
M5 - Performance	KPI Management, Employee Trackers, Review Validation.	15	Mediuml
M6 - PIM	Employee Administration, Profile Management, Attachments.	15	High
M7 - Recruitment	Candidate Management, Vacancy Assignment, Recruitment Validation.	15	High
M8 - Time	Attendance Reports, Employee Timesheets, Project Reports.	15	Medium

Detailed Module Coverage

M1 - Authentication

Validate secure access to the OrangeHRM application through positive and negative authentication scenarios. Coverage includes successful login validation, invalid credential handling, empty field validation, session verification, logout functionality, and unauthorized access prevention. These checks ensure that only authenticated users can access protected application resources.

M2 - Admin

Validate administrative workflows such as user search operations, role filtering, status filtering, employee selection, and user management activities. This module ensures that administrative users can perform configuration and management tasks effectively while maintaining system integrity.

M3 - Leave

Validate leave management workflows including leave requests, leave list searches, leave assignment activities, leave status filtering, and date validations. These scenarios help ensure accurate processing and management of employee leave information within the application.

M4 - My Info

Validate employee self-service functionalities such as personal information updates, contact details management, emergency contacts, qualifications, memberships, and other profile-related activities. This module ensures that employee information can be maintained accurately and consistently.

M5 - Performance

Validate employee performance management workflows including KPI tracking, employee trackers, reviewer assignments, and performance review activities. These validations help ensure that performance-related processes operate correctly and support organizational evaluation requirements.

M6 - PIM

Validate Personnel Information Management functionalities including employee creation, employee search operations, profile maintenance, job details management, document uploads, and employee record updates. This module acts as the central repository for employee-related information and therefore receives extensive automation coverage.

M7 - Recruitment

Validate recruitment workflows such as candidate registration, vacancy assignment, hiring manager selection, candidate searches, recruitment filters, and application status validations. These scenarios ensure that recruitment activities can be managed efficiently and accurately.

M8 - Time

Validate attendance and timesheet management activities including employee timesheets, attendance reports, attendance summaries, project reports, and employee-related reporting functionalities. These checks help ensure accurate workforce tracking and attendance management processes.

Types of Testing Planned

Testing Type	Purpose in This Project
Functional Testing	Confirms that each business workflow performs the expected behavior.
Validation Testing	Checks empty inputs, invalid inputs, and form-level validation messages.
UI Assertion Testing	Verifies headings, labels, buttons, status messages, and visible data.
Regression Testing	Re-runs stable automation suites after updates to ensure existing workflows continue to function correctly.
Session Testing	Checks application behavior after refresh and navigation activities
Workflow Testing	Validates complete end-to-end Human Resource Management processes.
Cross-Browser Testing	Executes tests in Chromium, Firefox, and WebKit browsers.

The automation framework incorporates multiple testing approaches to provide comprehensive validation of application behavior. Functional testing focuses on verifying business workflows, while validation testing ensures that the application responds appropriately to invalid or incomplete inputs.

Regression testing helps confirm that existing functionalities continue to operate correctly after updates or modifications. Cross-browser testing verifies consistent behavior across supported browsers, ensuring that users receive a reliable experience regardless of the browser being used.

The combination of these testing approaches improves overall test coverage and provides confidence in the stability and reliability of the OrangeHRM application.

Automation Approach

- Use Playwright with JavaScript for structured automation execution and clear assertion handling.Create separate spec files based on OrangeHRM modules, not random feature names.
- Maintain page classes for modules such as Authentication, Admin, Leave, My Info, Performance, PIM, Recruitment, and Time.
- Prefer stable Playwright locators such as `getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, and visible text-based locators.Avoid fragile selectors such as long XPath chains and unnecessary positional selectors unless there is no stable alternative.
- Use reusable test data files for employee names, login details, search inputs, leave data, and recruitment-related values.Capture screenshots, traces, and execution evidence only where needed so the report remains useful and not overloaded.

The automation approach focuses on building a maintainable framework that can be updated easily when the application changes. By organizing tests module-wise and using the Page Object Model, the project improves readability, reduces repeated code, and supports future expansion.

Planned Project Structure

Folder / File	Purpose
pages/auth.page.js	Authentication page actions such as login and logout handling.
pages/admin.page.js	Admin module actions including user search, role filter, and status filter handling.
pages/leave.page.js	Leave module actions such as leave list, apply leave, and assign leave workflows.
pages/myinfo.page.js	My Info module actions related to personal information and employee profile sections.
pages/performance.page.js	Performance module actions for KPI, tracker, and review-related validations.
pages/pim.page.js	PIM module actions for employee creation, search, and profile management.
pages/recruitment.page.js	Recruitment module actions for candidate and vacancy management workflows.
pages/time.page.js	Time module actions for timesheet, attendance, and report-related workflows.

tests/auth/auth.spec.js	Authentication module automation scenarios.
tests/admin/admin.spec.js	Admin module automation scenarios
tests/leave/leave.spec.js	Leave module automation scenarios.
tests/myinfo/myinfo.spec.js	My Info module automation scenarios.
tests/performance/performance.spec.js	Performance module automation scenarios.
tests/pim/pim.spec.js	PIM module automation scenarios.
tests/recruitment/recruitment.spec.js	Recruitment module automation scenarios.
tests/time/time.spec.js	Time module automation scenarios.
testData/	Reusable test data for login, employee, leave, recruitment, and search inputs.
playwright-report/	Generated Playwright HTML execution report after test execution.
reports/	Generated Allure and project execution report artifacts.
test-results/	Test execution artifacts such as screenshots, traces, and failure evidence.
playwright.config.js	Playwright configuration for browser projects, retries, timeouts, and reporters.
.github/workflows/	GitHub Actions workflow configuration for CI/CD execution.

The project structure is planned to keep automation files organized according to application modules. This makes the framework easier to understand, execute, debug, and extend. Separating test files, page objects, test data, and reports helps maintain a clean automation project suitable for future enhancements.

Test Data Strategy

Because **OrangeHRM** is a demo Human Resource Management application, the project must use safe, reusable, and replaceable test data. Tests should not depend on real employee information, confidential details, or permanent business records. The data strategy should support easy updates because employee names, dropdown values, and demo records may change over time.

- Use valid demo login credentials only for safe authentication testing.Use reusable employee names, search keywords, leave comments, and recruitment inputs from test data files.
- Avoid depending on a single employee record that may be modified or removed from the demo environment.Use configurable values for dropdown selections, date inputs, employee names, and vacancy-related fields.
- Maintain separate test data files so inputs can be updated without changing every test script.Avoid using real personal information, real employee data, or sensitive business details in automation scripts.

Risks and Mitigation

Risk	Mitigation
Dynamic UI changes	Use stable Playwright locators and keep selectors inside page objects.
Demo data changes	Use reusable and replaceable test data instead of depending on one fixed record.
Slow application response	Use Playwright auto-waiting, suitable timeouts, and retry configuration
Browser-specific failures	Execute scenarios across Chromium, Firefox, and WebKit to identify compatibility issues.
Locator failures	Prefer user-facing locators such as role, text, label, and placeholder-based selectors.
Session timeout or redirection	Validate login before module execution and handle page navigation carefully.
Report generation issues	Maintain proper Allure and Playwright report configuration in the project.

Reporting and Evidence Strategy

• **Playwright HTML Report** and **Allure Report** will be used as the primary execution reports for the automation framework.

• **Allure Reports** will provide execution summary, pass/fail status, duration, test details, screenshots, traces, and failure information.

• Screenshots will be collected for failed scenarios and important evidence points.

• Trace files will be retained for failed cases to support step-by-step debugging.

• GitHub Actions logs will serve as CI execution evidence when tests are triggered from the repository.

• A final summary will mention total planned scenarios, automated scenarios, pass count, fail count, browser coverage, and pending items.

Final Deliverables

Deliverable	Description
Capstone testing report	Complete document describing scope, modules, approach, risks, and deliverables.
Test case document	Module-wise test cases with expected results, priority, status, and regression coverage mapping.
Playwright automation project	JavaScript test scripts organized by OrangeHRM modules using Page Object Model.
Test data files	Reusable data for login, employee, leave, recruitment, search, and validation inputs.
Execution report	Playwright HTML Report and Allure Report showing test run details and final results
Evidence folder	Screenshots, traces, or CI logs supporting important passed and failed scenarios.
Final summary	Short summary of coverage, execution result, known limitations, and conclusion.

Expected Outcomes

The completed framework is expected to reduce repetitive manual verification, improve regression speed, provide better debugging evidence, and create reusable automation components for future expansion.

➢ Reusable **Playwright automation framework** using **Page Object Model**.

➢ Organized module-wise test scripts with maintainable structure.

➢ **Playwright HTML Reports** and **Allure Reports** for execution analysis.

➢ Screenshots, traces, and execution evidence for failures and validations.

➢ **GitHub Actions** workflow integration for automated execution.

➢ Test data files with configurable and reusable inputs.

➢ Cross-browser execution evidence across **Chromium, Firefox, and WebKit**.

➢ Structured automation coverage across **Authentication, Admin, Leave, My Info, Performance, PIM, Recruitment, and Time** modules.

Conclusion

This project demonstrates the implementation of a structured automation framework for a Human Resource Management platform using **Playwright** and JavaScript. The selected modules focus on realistic business workflows while maintaining safe testing boundaries and practical automation coverage.

The framework emphasizes maintainability through reusable components, stable locator strategies, modular design, and **Page Object Model** architecture. Reporting through **Playwright HTML Reports**, **Allure Reports**, screenshots, traces, and GitHub Actions execution logs improves execution visibility and debugging efficiency.

The completed automation suite is expected to reduce repetitive manual verification efforts, improve execution consistency, and strengthen overall testing reliability. Its modular approach also supports future scalability by enabling additional workflows and extended automation coverage.